

Masking Out of Order: Side-Channel Leaks from Software-Masked Cryptography on Out-of-Order Processors

Eden Desmet^{}, Suparna Kundu^{} and Ingrid Verbauwhede^{}

COSIC, KULeuven, Leuven, Belgium

`{eden.desmet, suparna.kundu, ingrid.verbauwhede}@esat.kuleuven.be`

Abstract. Masking, the primary countermeasure against differential power attacks, guarantees formal security under abstract execution models that are violated in modern micro-architectures. Meanwhile, processors with out-of-order micro-architectures are increasingly used for high-assurance tasks, yet their physical side-channel leakage remains poorly characterized, hindering side-channel security on such platforms. In this work, we present the first empirical study of physical power side-channel leakage on out-of-order cores. Through practical lab experiments, we identify and validate multiple micro-architectural leakage sources that undermine software masking: register renaming reintroduces register overwrites beyond software control; forwarding leaks through the common data bus, with less impact on security order than in-order forwarding; and concurrent instructions leaks through coupling, with affected instructions determined at runtime. We demonstrate that runtime scheduling and dynamic resource allocation undermine software-only mitigations. To address this, we propose countermeasures that shift part of the responsibility to hardware and require security by design. We further demonstrate that these effects are exploitable in practice by breaking the security of a theoretically secure software-masked lattice-based post-quantum implementation on an out-of-order core. Finally, we find that clock frequency significantly affects leakage of software-masked implementations. This makes security unstable across frequencies and suggests that cryptographic software should be constrained to verified frequencies.

Keywords: Side-Channel Analysis · Out-of-Order Execution · Micro-Architectures · Masking · Post-Quantum Cryptography

1 Introduction

Cryptographic implementations, in hardware *and* software, are vulnerable to *side-channel attacks* [KJJ99], which exploit data-dependent metadata, such as power consumption, to extract sensitive data. *Masking* [CJRR99, GP99, Cor99], the most prevalent countermeasure, randomizes sensitive values into multiple shares and operates on them separately. Masking is, under assumptions such as independent leakage of shares, theoretically secure.

Problematically, enforcing independent leakage of shares in physical implementations is nontrivial. In software, this challenge stems from unintended recombination of shares within the Central Processing Unit (CPU) micro-architecture.

For in-order cores, extensive prior work shows that recombination can occur due to micro-architectural effects such as overwrites in (pipeline) registers, memory and data buffers [BGG⁺15, MPW22, PV17], as well as combinatorial logic including address decoding and operand-forwarding multiplexers [GHP⁺21, GPM21]. Existing defenses rely primarily on software techniques, including register clearing or randomization, datapath flushing with

randomized operands, register partitioning into share domains, and instruction reordering [ZBPF18, GPM21, GD23]. This choice is pragmatic as software countermeasures are deployable on existing hardware and avoid performance penalties for unmasked code.

Out-Of-Order (OoO) platforms further complicate the landscape. They come with additional micro-architectural structures that can introduce leakage and reduce software control over execution behavior. Their performance features cause hardware to schedule instructions and allocate resources at runtime, so software cannot predict or control which share affects which resource or when this occurs.

Despite their seemingly obvious implications for side-channel leakage, the effects of OoO micro-architecture and their countermeasures have not been analyzed in detail. This is a pressing concern given the growing use of advanced processors in high-assurance tasks and the demonstrated potential for physical side-channel vulnerabilities to enable remote exploits [LKO⁺21, WPH⁺22], highlighting why side-channel security on OoO processors remains an open and relevant challenge.

Contributions Our main contributions are as follows:

- We identify micro-architectural behaviors in OoO cores such as register renaming, broadcasting, and parallel execution that violate the leakage-independence assumption underlying formal masking proofs, and we validate their leakage on real hardware.
- We demonstrate that these behaviors lead to exploitable side-channel leakages, through a practical attack on software-masked post-quantum cryptography with as few as 852 power traces.
- We show that first-order software-only masking is insufficient to guarantee first-order security on OoO cores, since software lacks control over the leaking resources. We therefore propose countermeasures that rely on hardware support.
- We find that running software-masked implementations at different clock frequencies on identical hardware changes leakage patterns, causing security instability.

2 Preliminaries

Notation Lowercase letters x denote variables, including polynomials. The $(i-1)$ -th secret share of x is written as x_{i-1} , and the set of all $d+1$ shares as $x_{\{0:d\}}$. The j -th bit of a variable is denoted by $x^{(j)}$; for polynomials, $x^{(j)}$ denotes the vector of the j -th bits of all coefficients. Bold lowercase letters $\mathbf{x}$ denote vectors, with elements accessed as $\mathbf{x}[k]$. Uppercase letters X denote matrices, with entries $X_{k,l}$. The m -th instance of a variable x is written as $x_{(m)}$. The ring of integers modulo q is denoted by $\mathbb{Z}_q$.

2.1 Masking

A masking scheme consists of (1) an encoding of the sensitive value s into a representation of $d+1$ uniformly distributed shares $(s_0, \dots, s_d)$ alongside (2) a transformation of the functionality of the unmasked operation to operations that calculate on the said shares.

Masking of d -th order ensures security against a d -th order attack under the probing model of [ISW03], as the measurement of at most d intermediate variables is independent of any sensitive variable s . As such, masking escalates the cost of side-channel attacks exponentially with the number of shares.

For instance, Boolean masking [CJRR99, GP99], a widely deployed and efficient masking type, encodes s as $s_0 \oplus \dots \oplus s_d$, where s_i denotes the $(i+1)$ -th share of s . First-order Boolean masking conceals the data with a single random mask, where s_1 is drawn from a uniform distribution and $s_0 = s \oplus s_1$.

2.2 Out-of-Order Execution in Hardware

OoO execution is a micro-architectural optimization that allows a processor to execute instructions in a different order than the program order, while producing the same architectural results. Instructions execute when their operands and resources are available, ensuring that data dependencies stall only the affected instructions rather than the entire pipeline.

OoO execution relies on traditional pipeline stages— Instruction Fetch (IF), Instruction Decode (ID), EXecute (EX) and RETirement (RET)—along with additional micro-architectural structures that track dependencies, schedule instructions dynamically and enforce in-order retirement (Fig. 1):

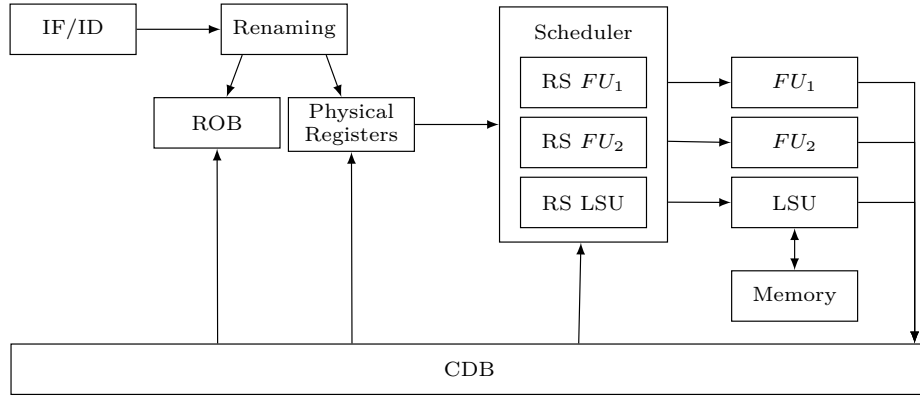


Figure 1: Simplified data flow in a generic OoO CPU micro-architecture, illustrating functional abstractions of hardware structures and their interactions.

- **Physical Registers** Each fetched instruction is assigned a physical register to store its results until retirement. Physical registers are implemented either as a dedicated register file or within the ReOrder Buffer (ROB). Register renaming maps program-visible architectural registers to available physical registers, allowing multiple in-flight versions of the same architectural register. Therefore, renaming defines where and for how long transient results are stored.
- **Reservation Stations** Renamed instructions are buffered in a Reservation Station (RS) until execution. A RS entry holds a decoded instruction along with tags identifying the pending operands, and in some micro-architectures, alongside available operands. Operand readiness is tracked via tags broadcast on the Common Data Bus (CDB). Once all operands are available, instructions become eligible for execution. Upon availability of an associated Functional Unit (FU), a dynamic scheduler selects one eligible RS entry for execution. Consequently, execution timing is a runtime decision dependent on data availability and resource contention.
- **Functional Units** FUs, such as Arithmetic Logic Units (ALUs) and Load-Store Units (LSUs), execute instructions in parallel. All results are broadcast on the CDB. Multiple FUs of the same type are typically present, making datapath allocation a runtime decision.
- **Common Data Bus** The CDB plays a role similar to forwarding logic in in-order cores by delivering results to dependent instructions without waiting for write-back. The CDB uses *shared* broadcast buses to transmit results and destination tags, which are matched by tag comparators in RS and ROB entries to detect operand

availability. Limited CDB bandwidth can delay broadcasts, causing results to be temporarily buffered in the EX stage, thereby extending the lifetime of transient data and influencing execution timing.

- **Reorder Buffer** The ROB tracks in-flight instructions and enforces in-order retirement, determining when speculative state becomes architectural and when physical registers are released.

2.3 Evaluation Setup

2.3.1 Target platform

We use the open-source, extensible RISC-V softcore *Proteus* [BNP23] (commit d48439e), which supports the RV32IM instruction set, as our baseline. Unless stated otherwise, it is configured as an OoO processor. We modified *Proteus* to support a configurable number of RS entries, register renaming with physical registers in the ROB, and a **trigger** instruction to mark program boundaries for trace alignment. The core is configured with 8 KiB of external memory, 64 ROB entries, and 3 LSUs; experiment-specific parameters, such as the number of ALUs or RS entries, are noted where relevant. Hardware implementations are deployed on a CW305-A100 Artix-7 Field-Programmable Gate Array (FPGA) board [New18].

The executed program is always preloaded in the instruction cache to simulate the ‘worst-case’ for side-channel analysis, *i.e.*, minimal fetch latency and thus maximal instruction interactions in the pipeline, while the data cache is disabled to ensure consistent memory latency. Deterministic dynamic scheduling—achieved via fixed scheduling algorithms, absence of an operating system and external events, and disabled data caches—ensures reproducible execution. The generation of all randomness and computation of secret sharings of inputs is done externally and pushed directly to the data memory of the FPGA. The 1 V power supply is *external* for stability. Traces were captured with a Tektronix DPO7254C oscilloscope at 20 GSamples/s.

Platform choice rationale Rather than characterizing a *specific* platform, our goal is to isolate leakage contributions of OoO features. *Proteus*’s modular design allows features to be incrementally enabled or disabled, enabling precise attribution of leakage to micro-architectural resources. While the core is not a high-end commercial processor, the micro-architectural features embody widely adopted OoO design principles, so the identified leakage mechanisms are expected to generalize across commercial OoO processors.

2.3.2 Test-Vector Leakage Assessment

To provide a clear indication of leakage of sensitive data or lack thereof in power traces, we use the Test-Vector Leakage Assessment (TVLA) methodology of [GJJR11, SM15]. TVLA uses Welch’s (two-tailed) t-test at each sample to determine if two sets or partitions of traces are distinguishable.

We introduce the following terminology: *first-order*, targeting the mean differences between partitions; *non-specific*, targeting all sensitive intermediate values at the same time without assumptions about how leakage occurs; and *fixed-versus-random*, one partition is associated with fixed inputs and one partition with randomly-varying inputs.

We adopt the standard threshold $t = \pm 4.5$ corresponding to a false-positive rate of $\sim 10^{-5}$ or confidence ~ 0.99999 ; t-statistics exceeding this in absolute value are considered significant.

2.3.3 Linear regression model

Complementing TVLA’s detection of leakage, we adopt a linear regression model—introduced by [GOP22] for leakage simulation—to identify specific intermediate values responsible for leakage and to quantify their individual contributions. The model $\mathcal{M}$ is estimated via *ordinary least squares* for each sample s in the Region-of-Interest (RoI) of the power trace. Formally, $\mathcal{M}$ can be expressed for all sample indices $s \in \{1, \dots, S\}$:

$$\underbrace{\begin{bmatrix} y[1] \\ y[2] \\ \vdots \\ y[k] \\ \vdots \\ y[N] \end{bmatrix}}_{N}^{(s)} = \underbrace{\begin{bmatrix} 1 & X_{:,1} & \cdots & X_{:,l} & \cdots & X_{:,P} \end{bmatrix}}_{N \times (P+1)} \underbrace{\begin{bmatrix} \beta[0] \\ \beta[1] \\ \vdots \\ \beta[l] \\ \vdots \\ \beta[P] \end{bmatrix}}_{(P+1)}^{(s)} + \underbrace{\begin{bmatrix} \varepsilon[1] \\ \varepsilon[2] \\ \vdots \\ \varepsilon[k] \\ \vdots \\ \varepsilon[N] \end{bmatrix}}_N^{(s)}$$

where:

- **Measurement vector** $\mathbf{y}_{(s)}$ contains N elements $y[k]_{(s)}$ representing the mean-centered power signal sampled at index s of the trace during execution $k \in \{1, \dots, N\}$.
- **Design matrix** X contains $P+1$ columns $X_{:,l}$: one intercept term and P independent variables representing leaking explanatory variables, e.g., the Hamming Weight (HW) of some intermediate data. Each row $X_{k,:}$ is constructed from the data provided to the target program during execution k . X is kept constant for all s .
- **Weight vector** $\beta_{(s)}$ contains $P+1$ weights corresponding to each independent variable. The weights vary with the sample s to capture time-dependent leakage.
- **Error vector** $\varepsilon_{(s)}$ captures residual noise at sample s , assumed to have constant, uncorrelated variance. Traces are mean-centered to remove drift and reduce autocorrelation. A large number of observations ($N \gg P$) ensures validity of asymptotic inference.

We are interested in the outcomes of the following statistical tests, described in Chapter 3 of [HTF09]:

- **Overall model significance F-test** assesses whether all independent variables collectively explain a significant portion of the variation in the dependent variable. Null Hypothesis (H_0): a model containing only the intercept ($X_{:,0}$) represents the data-dependent leakage. Alternative Hypothesis (H_A): the full model adds explanatory power. A p-value of the F-statistic smaller than the significance threshold indicates that the independent variables collectively capture meaningful leakage.
- **Nested F-test** evaluates whether a subset of L independent variables, with $1 \leq L < P$, adds explanatory power *beyond* a nested (restricted) model that excludes them. H_0 : a nested model $\mathcal{M}_{nested}$ where the corresponding columns are removed from X represents the data without loss. H_A : the subset in the the full (unrestricted) model $\mathcal{M}_{full}$ contributes. A p-value of the F-statistic smaller than the significance threshold indicates the excluded variables jointly provide significant additional data-dependent information.

We use a significance level of $p = 10^{-5}$ to control false positives across large trace sets and repeated tests, providing a uniform threshold that aligns the regression analysis with the confidence level used for TVLA.

3 Analysis of Leakage Sources

In this section, we examine each leakage source individually. For each, we explain the leakage mechanism, report our experiments and discuss targeted mitigations.

3.1 Register Renaming

Register overwrites in register files are a known source of transitional leakage [BGG⁺15, PV17]. This threatens software-masked implementations, since a masked value (e.g., $s \oplus m$) could be overwritten by another value using the same mask (m), causing Hamming Distance (HD) leakage that reveals $HD(s \oplus m, m) = HW(s)$. Software countermeasures, such as (1) statically partitioning registers into share domains and (2) randomizing intermediate registers, rely on direct access to physical registers. This control over physical registers is revoked in OoO cores with the introduction of register renaming. An instruction can be assigned to *any* free physical register, reintroducing transitional leakage due to register overwrites.

Evaluation We evaluate leakage introduced by register renaming by executing a program on two cores: a static core without renaming and a dynamic core with renaming. The program is a first-order software-masked **SecAnd** (Alg. 1), which masks the operation $z = x \wedge y$. To isolate leakage caused specifically by renaming rather than other reordering effects, the dynamic core is constrained to a single ALU with a one-entry RS.

The results in Fig. 2 reveal a clear distinction: while the static core exhibits no detectable leakage (Fig. 2c), the dynamic core shows a pronounced t-statistic peak (Fig. 2d). Regression model $\mathcal{M}$, with one non-constant independent variable $HW(z)$, shows a dependence on the unmasked result z in the same clock cycle, specifically when the second output share z_1 is written to its assigned physical register (Fig. 2b). This behavior stems from register renaming: after the instruction producing first output share z_0 (Line 2 in Alg. 1) commits, its physical register is freed and subsequently reused by the instruction generating second output share z_1 (Line 12 in Alg. 1). In contrast, the static core writes z_0 and z_1 to distinct architectural *and thus physical* registers, preventing such leakage.

Mitigation Robust mitigations against transitional leakage caused by register renaming must, at a high level, either reset or randomize registers, or partition the register file. As discussed and demonstrated for the masked **SecAnd**, software-only mitigations are insufficient to fully eliminate the leakage. In the presence of OoO execution, effective countermeasures therefore need to—at least partially—move from software into hardware. Disabling renaming altogether is also a possible mitigation; however, we do not consider this a practical solution due to its severe performance impact.

A first hardware-based mitigation is to clear or randomize physical registers immediately after they are freed, without requiring any software involvement. A second approach is to partition the physical register file into share domains. Software then tags each instruction with its share domain, and hardware constrains renaming to physical registers within the corresponding domain. Finally, a less elegant but practical measure is to pseudo-randomize physical register allocation. This ensures that transitional leakage varies across executions, thereby increasing the number of acquisitions required to observe exploitable leakage.

Conclusion This experiment shows that the set of established software-masking techniques which aim to avoid transitional leakages in register files hold up in static cores, and fail in cores with register renaming.

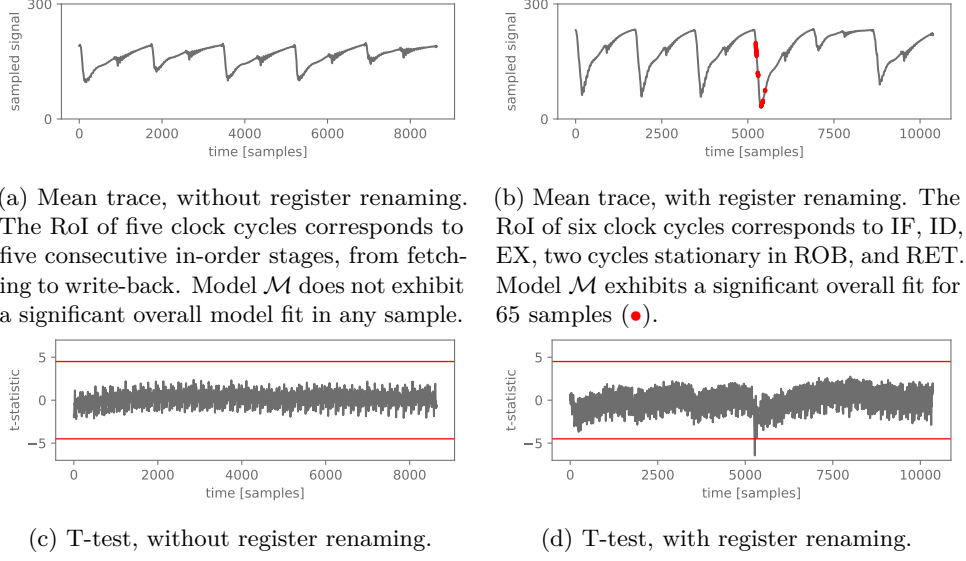


Figure 2: *Fixed-versus-random non-specific first-order* t-tests comparing execution with and without register renaming. **SecAnd** (Alg. 1) is executed on two platforms: without (left column) and with (right column) register renaming. In both cases, the RoI spans the complete execution of the instruction generating the second output share z_1 (Line 12 of Alg. 1). Input sets in both experiments are identical. The *fixed* partition holds unmasked operands x and y constant, while the *random* partition uses fully random 32-bit inputs. Each partition contains 50k traces. The clock frequency is 10 MHz. The significance threshold is indicated with red lines.

Algorithm 1 SecAnd from [CGV14, HKŠ]

Input 1: Two Boolean shares of a 32-bit value $x_{\{0,1\}}$ where $\bigoplus_{i=0}^1 x_i = x$
Input 2: Two Boolean shares of a 32-bit value $y_{\{0,1\}}$ where $\bigoplus_{i=0}^1 y_i = y$
Input 3: A 32-bit random value $r_{0,1} \leftarrow \{0,1\}^{32}$
Output: Two Boolean shares of a 32-bit value $z_{\{0,1\}}$ such that $\bigoplus_{i=0}^1 z_i = \bigoplus_{i=0}^1 x_i \wedge \bigoplus_{i=0}^1 y_i$

Require: Registers a_0, a_1 hold input shares x_0, x_1
Registers a_2, a_3 hold input shares y_0, y_1
Register t_1 holds random mask $r_{0,1}$
Registers t_2 – t_6 hold independent random 32-bit values

- 1: **and** t_0, a_0, a_2 ▷ $x_0 \wedge y_0$
- 2: **xor** a_5, t_0, t_1 ▷ Generate first output share $z_0 = (x_0 \wedge y_0) \oplus r_{0,1}$
- 3: **and** a_6, a_0, a_3 ▷ $x_0 \wedge y_1$
- 4: **xor** a_6, a_6, t_1
- 5: **add** $t_1, t_2, zero$ ▷ Randomize reused register
- 6: **and** t_1, a_1, a_2 ▷ $x_1 \wedge y_0$
- 7: **xor** a_6, t_1, a_6 ▷ $r_{1,0} = (x_1 \wedge y_0) \oplus (x_0 \wedge y_1) \oplus r_{0,1}$
- 8: **add** $t_1, t_3, zero$
- 9: **add** t_1, t_4, t_5 ▷ Execute of instruction with random operands
- 10: **add** $t_1, t_6, zero$
- 11: **and** t_1, a_1, a_3 ▷ $x_1 \wedge y_1$
- 12: **xor** a_6, a_6, t_1 ▷ Generate second output share $z_1 = (x_1 \wedge y_1) \oplus r_{1,0}$

3.2 Forwarding

In superscalar in-order processors, forwarding happens through multiplexers which pick one out of multiple forwarded results as the operand of a FU. Their combinatorial select signal can glitch, causing multiple shares from different instructions to recombine in the same cycle [GPM21], which is a major source of leakage.

OoO cores handle operand delivery differently. Results are broadcast via the CDB and selectively captured, either in a centralized physical register file or in RS entries, by instructions waiting for tag matches. Operand capture is thus a register write gated by tag comparison, which provides *stable* operands to the EX stage. This absence of glitching eliminates the primary leakage mechanism caused by forwarding in superscalar in-order pipelines.

Evaluation Any leakage attributable to operand capture in an OoO core would have to manifest as additional switching activity *beyond* that induced by the CDB broadcast itself. We test this by comparing CDB broadcasts of identical values under two conditions: (1) broadcasts not consumed by any waiting instruction (*non-capture*), and (2) broadcasts captured by a dependent instruction in the RS (*capture*). The program, whose schedule is shown in Fig. 3a, is run on an OoO core with two parallel ALUs, each with a two-entry RS, and a central 32-bit CDB. A single-word CDB is used to avoid potential interactions between words on the bus.

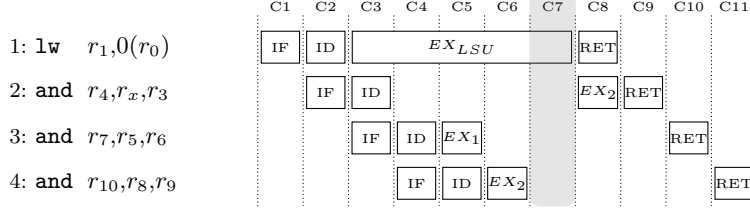
The TVLA (Fig. 3b) shows no significant difference between *capture* and *no-capture* conditions, indicating operand capture adds no measurable leakage beyond the CDB broadcast.

While it is well-known that *any* data bus can leak via switching activity, the CDB is a notable case: its large capacitive load—with fan-out including all RS entries, ROB entries, and physical registers—makes sequential writes highly visible. Additionally, contention on the CDB can reorder broadcasting relative to the execution order, causing a new set of recombinations of data.

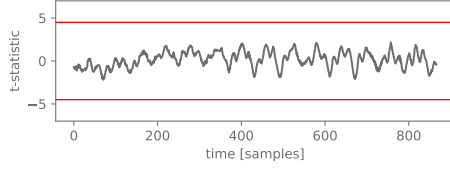
Regression-based analysis confirms that consecutive writes on the CDB are highly visible on our platform. The overall fit of $\mathcal{M}$, with one non-constant independent variable $HD(cdb_5, cdb_6)$ representing broadcasts on clock cycle 5 (cdb_5) and 6 (cdb_6), yields p-values as low as 10^{-50} (Fig. 3c). We argue that the large capacitive load of the CDB makes the significance of this leakage highly transferable to other platforms.

Mitigation Mitigation efforts for forwarding leakage should focus on the CDB. Preventing leakage from forwarding requires that the results from one share domain do not interact with operands from another share domain. In in-order processors, this can be enforced through software-level instruction separation [GPM21], which prevents overlapping shares in a single cycle. This approach does not extend to OoO processors, where execution order depends on dynamic factors such as program data dependencies, RS and FU occupancy, variable execution latencies, and contention on the CDB. Guaranteeing no conflicts of share domains in the worst case would require impractically large instruction gaps. Consequently, handling these dynamic effects requires hardware support. One approach is a share-aware micro-architecture with a *distributed* CDB, instead of a central or shared one, to isolate forwarding paths across share domains.

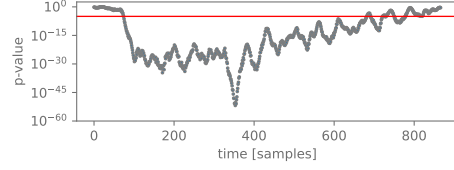
Conclusion In contrast to research on superscalar in-order cores, our measurements indicate that any switching activity associated with operand capture falls below the noise floor. It appears to be negligible compared to the leakage induced by the CDB itself. Any mitigation effort related to forwarding should therefore focus on the CDB rather than on operand capture logic.



(a) Scheduling diagram showing program executed on an OoO core with two parallel ALUs, each with a two-entry RS. The RoI, indicated with a grey box, spans one clock cycle where instruction 1 broadcasts data (r_x) on the CDB. For the *capture* partition, $x = 1$: instruction 2 has a data dependency on instruction 1 and captures the broadcast data. For the *non-capture* partition, $x = 2$: the broadcast data is not captured by instruction 2. For the *non-capture* partition, without data dependencies, we create identical control flow to the *capture* partition by causing contention on the ALUs by filling the RS entries with random instructions. The subscript on EX stage refers to the FU on which it is executing, namely ALU₁, ALU₂, or an LSU. An EX stage ends when the result is broadcast on the CDB.



(b) *Non-specific first-order* t-test comparing forwarding with (*capture*) and without operand capture (*non-capture*). The input sets in both partitions are identical, with all operands being randomized 32-bit values. Each partition contains 50k traces.



(c) F-test p-values on overall fit of regression model $\mathcal{M}$ for all samples in the RoI. The set contains 50k traces of the *non-capture* partition.

Figure 3: Evaluation of leakage related to forwarding. **Top row:** The diagram shows the scheduling of the executed program and indicates how the program differs for the *capture* partition, meaning forwarding with operand capture, and the *non-capture* partition, forwarding without operand capture. **Bottom row:** The statistical tests examine leakage related to forwarding. The traces are aligned on the RoI indicated in the scheduling diagram. The clock frequency is 20 MHz. The significance thresholds are indicated with a red line.

3.3 Parallel Execution

Coupling effects—arising from signal interaction (*crosstalk*) and shared power networks (*power supply noise*)—are widely known to reduce the side-channel security of hardware-masked implementations [DFS15, DBG⁺17, DEM18]. However, prior work on micro-architectural leakage has predominantly relied on simulated traces, which inherently omit such physical effects. In the following, we investigate whether coupling manifests in practice for software running on a CPU, by analyzing physical side-channel traces.¹

Evaluation To identify measurable coupling interactions, the experiment focuses on the EX stage as its parallel FUs (1) actively process data and (2) naturally introduce neighboring wiring and shared power/ground networks. For this scenario, we use an OoO core with two parallel ALUs (ALU₁ and ALU₂), each with a one-entry RS. We evaluate whether two concurrent `add` operations produce additional significant data-dependent behavior due to inter-ALU coupling, beyond the effects of standard switching activity or glitches. To detect and characterize the coupling effect, we construct regression models and perform statistical tests:

- We define an unrestricted regression model, $\mathcal{M}_{full}$, that includes independent variables for both standard switching activity through HD and simultaneous signal presence, reflecting energy scaling with the product of voltage signals such as in coupling, through bitwise products between operands across ALUs. Specifically, for each operand in ALU₁ ($operand_{1,A}$ and $operand_{1,B}$) combined with each operand in ALU₂ ($operand_{2,A}$ and $operand_{2,B}$), $\mathcal{M}_{full}$ contains:
 - A variable of the form $HW(operand_1 \oplus operand_2) = HD(operand_1, operand_2)$ (switching activity).
 - A variable of the form $HW(operand_1 \wedge operand_2)$ (simultaneous signal).

Samples modeled by these independent variables in which $operand_1$ and $operand_2$ are shares of the same native value violate leakage independence of shares. Test overall fit of $\mathcal{M}_{full}$ in Fig. 4a reveals that many samples depend on these independent variables, which hence jointly describe a leakage source underlying those samples. This leakage source cannot be explained by activity between a single ALU, as all independent variables combine one input from an ALU with one input from another ALU. Furthermore, Fig. 4a shows that this leakage effect is distributed rather than time-specific, which would be the case for leakage related to glitches or overwrites.

- We define a nested model, $\mathcal{M}_{nested}$, which includes only the four switching activity variables ($HD(operand_1, operand_2)$). Comparing $\mathcal{M}_{full}$ to $\mathcal{M}_{nested}$ in Fig. 4b shows that the bitwise product variables provide explanatory power *beyond* the nested model. In other words, variables representing coupling capture additional data-dependent information beyond the standard switching activity.
- We create models $\mathcal{M}_{full,m}$ for $m \in \{0, \dots, 31\}$, which are copies of $\mathcal{M}_{full}$ apart from modifying the bitwise product terms to the form $HW(operand_1 \wedge ROR_m(operand_2))$, where ROR_m is a circular shift to the right with m bits.² Comparing each $\mathcal{M}_{full,m}$ to $\mathcal{M}_{nested}$ in Fig. 4c shows similar explanatory power for all m , indicating that the leakage effect is robust to bit permutation. This rules out effects tied to exact bit identities, such as registers overwrites, logic reuse or glitch-based leakage.

¹We highlight that (1) routing/placement was not adjusted nor were instructions iterated to amplify couplings, (2) the measurement setup was not modified for external amplification [LBS19], and (3) an external power supply was used for stability.

²Note that $\mathcal{M}_{full,m=0}$ equals $\mathcal{M}_{full}$.

Collectively, these observations suggest that the measured data-dependent behavior arises from inter-ALU coupling, rather than from switching activity or glitches.

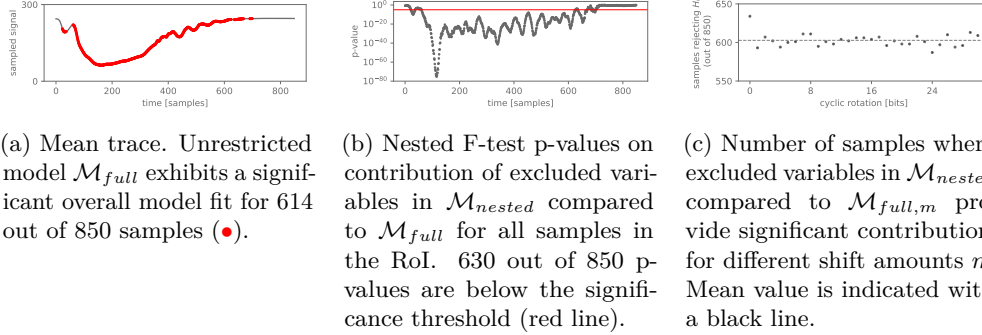


Figure 4: Regression-based F-tests under concurrent operation. Two **add** operations are executed in parallel on two ALUs. The RoI spans the clock cycle where both **add** operations are simultaneously in the EX stage. The set consists of 5k traces where all consumed operands are randomized 32-bit values. The clock frequency is 20 MHz.

Mitigation Coupling leakage in parallel ALUs can be reduced by preventing concurrent execution of instructions operating on shares of the same sensitive value. In in-order pipelines, this can be enforced in software by separating such instructions in program order, as shown in [GPM21]. In contrast, OoO pipelines schedule instructions dynamically in hardware, so software-level instruction placement alone is insufficient. In this case, mitigation requires hardware support. For example, the dynamic scheduler can be made aware of share domains through annotated instructions and enforce constraints that prevent co-residency of conflicting instructions on parallel FUs.

Conclusion This experiment demonstrates statistically significant coupling leakage. While this leakage need not manifest on all platforms, the presence of parallel FUs in OoO and superscalar processors enables it in principle, and its existence can only be established on representative physical hardware. Consequently, the consideration of coupling leakage should happen at design time. These findings motivate hardware-based side-channel assessments over simulated traces, especially as higher frequencies increase simultaneous switching and reduce settling time which make coupling-induced leakage more observable.

4 Clock Frequency as Side-Channel Parameter

Moradi et al. [MM13] showed that in synchronous hardware-masked designs, higher clock frequencies can cause analog summation of power consumption from consecutive cycles, recombining the leakage of different shares even when only one share is active per cycle.

We extend this observation of frequency-induced leakage to a distinctly different execution environment: software-masked implementations on a CPU pipeline. Unlike hardware masking, multiple instructions are active in the same cycle, so leakage can increase not only due to analog summation but also because of changes in *architectural overlap*, *i.e.*, the temporal overlap of instructions from different shares. Adjusting the frequency can change instruction timing, which in turn affects each instruction’s contribution to each power sample.

In the following, we ask how different fixed clock frequencies impact observable power side-channel leakage on identically synthesized CPUs.

Evaluation We execute Alg. 1 (Alg. 1) at fixed clock frequencies 5 MHz, 10 MHz, and 40 MHz on an OoO core.

The TVLAs (Figs. 5d to 5f) show that the magnitude and shape of the t-statistics depend on the frequency. No significant samples appear at 5 MHz, while more significant samples emerge at 10 MHz and 40 MHz. The change in the shape (rather than amplitude alone) of the t-statistics across frequencies suggests that different instructions contribute to each sample at varying proportions, providing empirical evidence that the leakage we observe is amplified by *architectural overlap* rather than by analog summation alone.

We complement the TVLA with a linear regression model $\mathcal{M}$ using $HW(x)$ and $HW(y)$ as independent variables, with x and y the unmasked operands. The overall model fit per sample (Figs. 5g to 5i) follows the same trend as the TVLA results. In addition, the regression reveals that measurements at higher frequencies yield smaller p-values for most samples, indicating stronger statistical confidence in data-dependent leakage.

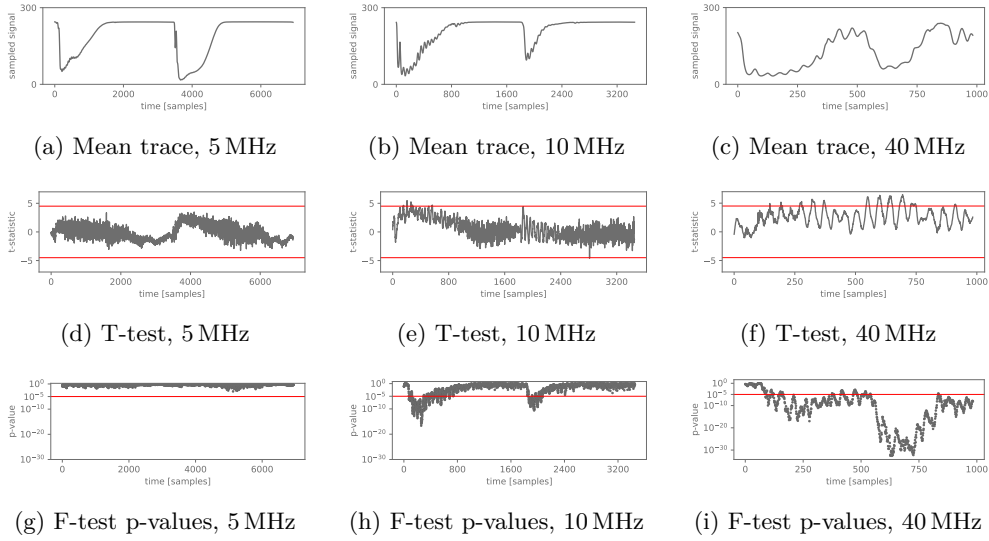


Figure 5: Statistical tests comparing leakage for different clock frequencies. **SecAnd** (Alg. 1) is executed at 5 MHz (left column), 10 MHz (middle column), and 40 MHz (right column). All experiments use a OoO core with two ALUs, each a dual-entry RS, and single-word CDB (including identical hardware synthesis) and identical sampling frequency at 20 GHz. **Top row:** Mean traces indicate the temporal structure of the measurements. No specific leakage source is targeted; the RoI contains two consecutive clock cycles which could leak unmasked values: the first spans $x_0 \wedge y_0$ (Line 1) and $x_0 \wedge y_1$ (Line 3) concurrently in EX, with Line 1 broadcast on the CDB; the second spans Line 3 broadcast and Line 1 commit. **Middle row:** *Fixed-versus-random non-specific first-order* t-tests. Input sets in all experiments are identical. The *fixed* partition holds unmasked operands x and y constant, while the *random* partition uses fully random 32-bit inputs. Each partition contains 20k traces. Significance thresholds are indicated by red lines. **Bottom row:** F-test p-values for the overall fit of regression model $\mathcal{M}$ for all samples in the RoI, fit using 20k traces of the *random* partition. The significance threshold is indicated by a red line.

Conclusion Clock frequency has a direct impact on side-channel leakage; results observed at one frequency do not trivially generalize to others. A configuration that appears secure at one frequency may exhibit different leakage or even become vulnerable at another. On modern CPUs, where frequency is changed at runtime to manage power consumption, this introduces security instability. Consequently, cryptographic operations should be executed only at fixed frequencies that have been individually verified for side-channel resistance.

5 Case Study

In this section, we show that leakages from OoO processors are exploitable, by extracting information about the secret key of masked lattice-based post-quantum Key-Encapsulation Mechanisms (KEMs).

Masked lattice-based KEMs Lattice-based KEMs, such as Saber [KDvB⁺22] and Kyber [DvBV23] have three algorithms: (1) **Key-Generation**, (2) **Encapsulation**, and (3) **Decapsulation**. **Decapsulation** is most attractive from a side-channel attack perspective, as it uses a long-term secret key ($\mathbf{s}$) during the **Decryption** procedure. The **Decapsulation** operation has three main steps: (1) **Decryption**, (2) **Re-Encryption**, and (3) **Ciphertext Comparison**. In this attack, we target the masked **Ciphertext Comparison** step (Alg. 2), specifically the **BooleanAllBitsOneTest** algorithm (Alg. 3), which tests whether all the unmasked bits are 1 or not.

Attack overview We follow the attack of [BDH⁺21], which slightly alters the ciphertext and relies on a **plaintext-checking** oracle to indicate whether the decrypted message (m') during **Decapsulation** matches the original message (m) from **Encapsulation**. In the following, we briefly outline the attack and the role of the **plaintext-checking** oracle.

Assume $c_{\text{pub}} = (\mathbf{u}_{\text{pub}}, v_{\text{pub}})$ is a public ciphertext generated from the encryption using message m . First, we *slightly* modify the ciphertext $c'_{\text{pub}} = (\mathbf{u}_{\text{pub}}, v'_{\text{pub}})$, where $v'_{\text{pub}} = v_{\text{pub}} + Ex^i$, $E \in \mathbb{Z}_t$, and $i \in \{0, 1, \dots, n-1\}$. Without loss of generality, we can fix $i = 0$ such that $v'_{\text{pub}} = v_{\text{pub}} + E$. Second, we perform **Decapsulation** using c'_{pub} as input. Third, we observe whether the introduced error E in the modified ciphertext changes the decrypted message (m') from the original message (m) during the **Decryption** step. There are two possible outcomes, which are distinguished using the **plaintext-checking** oracle:

1. $m' = m$: The introduced error E does not change the original message. In this case, the unmasked re-encrypted ciphertext $c_{\text{re}} = (\mathbf{u}_{\text{re}}, v_{\text{re}})$ should be identical with $c_{\text{pub}} = (\mathbf{u}_{\text{pub}}, v_{\text{pub}})$. Then the difference between the input ciphertext ($c'_{\text{pub}} = (\mathbf{u}_{\text{pub}}, v'_{\text{pub}})$) and the re-encrypted ciphertext would be only at the $i = 0$ th coefficient of v'_{pub} and v_{re} ($= v_{\text{pub}}$).
2. $m' \neq m$: The introduced error E changes the original message. In this case, the unmasked value of the re-encrypted ciphertext c_{re} will be significantly different from c_{pub} and the modified public ciphertext input of the **Decapsulation** operation, c'_{pub} .

Finally, we try to find such an error value E that satisfies Case 2 (i.e., introduced error E changes the decrypted message from the encrypted one $m' \neq m$), but $E - 1$ satisfies Case 1 (i.e., $m' = m$). From this, we collect the following approximate equalities, which are related to the secret key ($\mathbf{s}$).

$$\lfloor \frac{q}{t}(\frac{q}{4} - E) \rfloor \leq (v_{\text{pub}} - \frac{q}{2}m - \mathbf{s}^T \mathbf{u}_{\text{pub}})[0] < \lfloor \frac{q}{t}(\frac{q}{4} - (E - 1)) \rfloor \quad (\text{since } i = 0) \quad (1)$$

$$\implies (v_{\text{pub}} - \frac{q}{2}m - \mathbf{s}^T \mathbf{u}_{\text{pub}})[0] \approx \lfloor \frac{q}{t}(\frac{q}{4} - E + \frac{1}{2}) \rfloor \quad (2)$$

Here q is the modulus of the underlying lattice learning with errors or learning with rounding hard problem. By collecting several such secret key-related equations (Eq. (1) or Eq. (2)), we can either recover the full secret key or drastically reduce the security of the scheme, depending on the KEM scheme and parameter choices. For more details regarding the attack procedure, we refer to [BDH⁺21].

Construction of plaintext-checking oracle We elaborate on how to construct the plaintext-checking oracle by collecting side-channel information from an OoO CPU. We target the `BooleanAllBitsOneTest` algorithm (Alg. 3) of the masked `Ciphertext Comparison` step of the Decapsulation operation (Alg. 2).

- **If the error E falls in Case 1 (i.e., $m' = m$),** then all but one coefficient of the unmasked re-encrypted ciphertext c_{re} and modified input ciphertext c'_{pub} match. This implies:
 - All $\log_2 p$ bits of all coefficients of the unmasked l -size vector of $(n - 1)$ -degree polynomials $\mathbf{u}$ (Input 1 of Alg. 3) are 1.
 - All $\log_2 t$ bits of all but one coefficient of the unmasked $(n - 1)$ -degree polynomial v (Input 2 of Alg. 3) are 1.
 - One coefficient of v has at least one bit out of $\log_2 t$ bits (0th bit as $i = 0$) as 0.

Consequently, at least once the `SecAnd` operation in Line 11 of Alg. 3 is executed with a *structured* input set, where the two unmasked n -bit operands satisfy one of the following conditions: (a) both operands have their first bit equal to 0 and all remaining bits equal to 1, or (b) exactly one operand has its first bit equal to 0 and all remaining bits equal to 1, while the other operand has all bits equal to 1.

- **Otherwise, the error E falls in Case 2 (i.e., $m' \neq m$),** then most of the coefficients of the unmasked re-encrypted ciphertext c_{re} and modified input ciphertext c'_{pub} will not match. This implies that all coefficients of the unmasked $\mathbf{u}$ and v are uniformly random $\log_2 p$ - and $\log_2 t$ -bit values, respectively. Consequently, the `SecAnd` in Line 11 of Alg. 3 operates on a *random* input set, consisting of two uniformly random unmasked n -bit operands.

The construction above reduces the problem of distinguishing Case 1 from Case 2 to distinguishing executions of the `SecAnd` operation with either a *structured* or a *random* input set. We assess the distinguishability of these two execution classes by performing a TVLA on the `SecAnd` operation (Alg. 1) executed on an OoO processor, where the *structured* and *random* input sets define the two partitions. As shown in Fig. 6b, the mean power traces of the two partitions exhibit a significant difference at multiple samples over the course of the execution. Furthermore, Fig. 6c shows that this difference becomes reliably statistically significant with as few as 426 traces per partition (852 traces total). This observable difference in means therefore provides a reliable distinguisher between Case 1 and Case 2, and directly instantiates a **plaintext-checking oracle**.

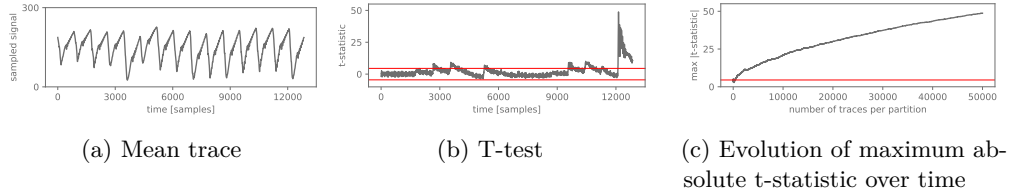


Figure 6: *Non-specific first-order TVLA* comparing execution of Case 1 (i.e., $m' = m$) and Case 2 (i.e., $m' \neq m$). `SecAnd` (Alg. 1) is executed with two distinct input sets: the first partition sets the two unmasked inputs as described for Case 1 (i.e., $m' = m$) and the second partition sets them as random 32-bit values as described for Case 2 (i.e., $m' \neq m$). Each partition contains 50k traces. The RoI spans the complete execution of the `SecAnd` (Alg. 1), from clock cycle one representing Line 1 in IF and clock cycle 15 representing Line 12 in RET. The clock frequency is 20 MHz. The significance thresholds are indicated with red lines.

Algorithm 2 Masked Comparison, from [KDvB⁺22]

Input 1: $(d + 1)$ Boolean shares of a l -size vector of $(n - 1)$ -degree polynomials, $\mathbf{u}_{\mathbf{re}\{0:d\}}$, with coefficients in $\mathbb{Z}_p$ ($p = 2^{\epsilon_p}, \epsilon_p \in \mathbb{N}$), and $\bigoplus_{i=0}^d \mathbf{u}_{\mathbf{re},i} = \mathbf{u}_{\mathbf{re}}$

Input 2: $(d + 1)$ Boolean shares of a $(n - 1)$ -degree polynomial $v_{\mathbf{re}\{0:d\}}$, with coefficients in $\mathbb{Z}_t$ ($t = 2^{\epsilon_t}, \epsilon_t \in \mathbb{N}$), and $\bigoplus_{i=0}^d v_{\mathbf{re},i} = v_{\mathbf{re}}$

Input 3: Public l -size vector of $(n - 1)$ -degree polynomials, $\mathbf{u}_{\mathbf{pub}}$, coefficients in $\mathbb{Z}_p$

Input 4: Public $(n - 1)$ -degree polynomial, $v_{\mathbf{pub}}$, with coefficients in $\mathbb{Z}_t$

Output: One bit number, b , such that $b = 1$ iff $c_{\mathbf{re}} (= (\mathbf{u}_{\mathbf{re}}, v_{\mathbf{re}}))$ equals $c_{\mathbf{pub}} (= (\mathbf{u}_{\mathbf{pub}}, v_{\mathbf{pub}}))$, else $b = 0$

- 1: $\mathbf{u}_{\{0:d\}} \leftarrow \mathbf{u}_{\mathbf{re}}$
- 2: $\mathbf{u}_1 \leftarrow \mathbf{u}_1 \oplus \mathbf{u}_{\mathbf{pub}}$
- 3: $v_{\{0:d\}} \leftarrow v_{\mathbf{re}}$
- 4: $v_1 \leftarrow v_1 \oplus v_{\mathbf{pub}}$
- 5: $\mathbf{u}_1 \leftarrow \neg \mathbf{u}_1$
- 6: $v_1 \leftarrow \neg v_1$
- 7: $b_{\{0:d\}} \leftarrow \text{BooleanAllBitsOneTest}(\mathbf{u}_{\{0:d\}}, v_{\{0:d\}})$
- 8: $\text{unmask}(b)$
- 9: **return** b

6 Conclusion and Future Work

We studied the impact of out-of-order execution on power side-channel leakage through lab experiments. Our results revealed leakage sources inherent to out-of-order micro-architectures. In particular, register renaming abstracts architectural from physical registers, reintroducing overwrites and leaking in-flight results even when software accounts for them. We also identified coupling between concurrent instructions as a significant leakage source, highlighting that side-channel security requires validation on real hardware. We found that leakage sources from in-order cores do not necessarily transfer to out-of-order designs. For instance, operand capture in out-of-order processors is realized as a gated register write rather than combinatorial logic, naturally avoiding the substantial security order reductions caused by glitchy operand-selecting multiplexers in in-order forwarding.

We identified operating frequency as a critical factor that can qualitatively alter side-channel security conclusions on the same hardware. A software-masked implementation that appears secure at one frequency may exhibit different leakage or even become vulnerable at another, due both to analog summation of power across consecutive cycles and shifts in the real-time alignment of instructions. On modern CPUs, where frequency can change at runtime to manage power, this introduces security instability. Consequently, frequency should be treated as a first-class evaluation parameter, and secure deployment may require constraining execution to verified fixed frequencies.

Overall, our findings demonstrate that established software-masking techniques are insufficient for mitigating micro-architectural leakage on out-of-order cores, where instruction scheduling and datapath allocation are dynamically determined by hardware. Effective masking therefore requires hardware support and must be addressed at design time. Finally, we proposed targeted mitigations with hardware support for the observed leakage sources that protect masked code without affecting unprotected implementations. We motivate future research on hardware-supported software masking. Such approaches may emphasize greater software control over micro-architectural resources [KS20, GMPP20] or shift more responsibility to hardware, e.g., through cryptographic instruction set extensions that specify operand share domains and enforce their separation in hardware. Moreover, the abundance of replicated resources in modern processors offers opportunities for low-overhead mitigations through physical separation of shares.

Algorithm 3 BooleanAllBitsOneTest, from [KDvB⁺22]

Input 1: $(d+1)$ Boolean shares of a l -size vector of $(n-1)$ -degree polynomials, $\mathbf{u}_{\{0:d\}}$, with coefficients in $\mathbb{Z}_p$ ($p = 2^{\epsilon_p}, \epsilon_p \in \mathbb{N}$), and $\bigoplus_{i=0}^d \mathbf{u}_i = \mathbf{u}$

Input 2: $(d+1)$ Boolean shares of a $(n-1)$ -degree polynomial $v_{\{0:d\}}$, with coefficients in $\mathbb{Z}_t$ ($t = 2^{\epsilon_t}, \epsilon_t \in \mathbb{N}$), and $\bigoplus_{i=0}^d v_i = v$

Output: $(d+1)$ Boolean shares of one-bit number, $b_{\{0:d\}}$, with $b_i \in \{0, 1\}$ $\forall i \in \{0, \dots, d\}$ and $\bigoplus_{i=0}^n b_i = b$. $b = 1$ iff all the bits of every coefficient of $\mathbf{u}$ and v are 1, else $b = 0$

```

1: for  $j = 1$  to  $l$  do
2:   for  $s = 1$  to  $\epsilon_p$  do
3:      $\mathbf{y}[s][j]_{\{0:d\}} \leftarrow \text{Bitslice}(\mathbf{u}^{(s)}[j]_{\{0:d\}})$ 
4:   end for
5: end for
6: for  $s = 1$  to  $\epsilon_t$  do
7:    $x[s]_{\{0:d\}} \leftarrow \text{Bitslice}(v_{\{0:d\}}^{(s)})$ 
8: end for
9:  $w_{\{0:d\}} \leftarrow y[1]_{\{0:d\}}$ 
10: for  $s = 2$  to  $\epsilon_t$  do
11:    $w_{\{0:d\}} \leftarrow \text{SecAnd}(w_{\{0:d\}}, x[s]_{\{0:d\}})$ 
12: end for
13: for  $j = 1$  to  $l$  do
14:    $\mathbf{y}'[j]_{\{0:d\}} \leftarrow \mathbf{y}[1][j]_{\{0:d\}}$ 
15:   for  $s = 2$  to  $\epsilon_p$  do
16:      $\mathbf{y}'[j]_{\{0:d\}} \leftarrow \text{SecAnd}(\mathbf{y}'[j]_{\{0:d\}}, \mathbf{y}[s][j]_{\{0:d\}})$ 
17:   end for
18:    $w_{\{0:d\}} \leftarrow \text{SecAnd}(w_{\{0:d\}}, \mathbf{y}'[j]_{\{0:d\}})$ 
19: end for
20: for  $j = \log_2 n - 1$  to  $0$  do
21:    $w'_{\{0:d\}} \leftarrow w_{\{0:d\}} \gg 2^j$ 
22:    $w_{\{0:d\}} \leftarrow w_{\{0:d\}} \bmod (2^{2^j})$ 
23:    $w_{\{0:d\}} \leftarrow \text{SecAnd}(w_{\{0:d\}}, w'_{\{0:d\}})$ 
24: end for
25:  $b_{\{0:d\}} \leftarrow w_{\{0:d\}}$ 
26: return  $b_{\{0:d\}}$ 

```

Acknowledgements

This work has been partially supported by the Horizon 2020 ERC Advanced Grant (101020005 Belfort), the Cybersecurity Research Program Flanders (CRPF) with reference number VOEWICS02, the Belgian-QCI (3E230370) (see beqci.eu), and Intel Corporation.

References

- [BDH⁺21] Shivam Bhasin, Jan-Pieter D’Anvers, Daniel Heinz, Thomas Pöppelmann, and Michiel Van Beirendonck. Attacking and defending masked polynomial comparison. *IACR TCHES*, 2021(3):334–359, 2021.
- [BGG⁺15] Josep Balasch, Benedikt Gierlichs, Vincent Grosso, Oscar Reparaz, and François-Xavier Standaert. On the cost of lazy engineering for masked software implementations. In Marc Joye and Amir Moradi, editors, *Smart Card Research and Advanced Applications*, pages 64–81, Cham, 2015. Springer International Publishing.
- [BNP23] Marton Bognar, Job Noorman, and Frank Piessens. Proteus: An extensible risc-v core for hardware extensions. In *RISC-V Summit Europe ’23*, June 2023. Commit d48439e.
- [CGV14] Jean-Sébastien Coron, Johann Großschädl, and Praveen Kumar Vadnala. Secure conversion between Boolean and arithmetic masking of any order. In Lejla Batina and Matthew Robshaw, editors, *CHES 2014*, volume 8731 of *LNCS*, pages 188–205. Springer, Berlin, Heidelberg, September 2014.
- [CJRR99] Suresh Chari, Charanjit S. Jutla, Josyula R. Rao, and Pankaj Rohatgi. Towards sound approaches to counteract power-analysis attacks. In Michael J. Wiener, editor, *CRYPTO’99*, volume 1666 of *LNCS*, pages 398–412. Springer, Berlin, Heidelberg, August 1999.
- [Cor99] Jean-Sébastien Coron. Resistance against differential power analysis for elliptic curve cryptosystems. In Çetin Kaya Koç and Christof Paar, editors, *CHES’99*, volume 1717 of *LNCS*, pages 292–302. Springer, Berlin, Heidelberg, August 1999.
- [DBG⁺17] Thomas De Cnudde, Begül Bilgin, Benedikt Gierlichs, Ventzislav Nikov, Svetla Nikova, and Vincent Rijmen. Does coupling affect the security of masked implementations? In Sylvain Guilley, editor, *COSADE 2017*, volume 10348 of *LNCS*, pages 1–18. Springer, Cham, April 2017.
- [DEM18] Thomas De Cnudde, Maik Ender, and Amir Moradi. Hardware masking, revisited. *IACR TCHES*, 2018(2):123–148, 2018.
- [DFS15] Alexandre Duc, Sebastian Faust, and François-Xavier Standaert. Making masking security proofs concrete - or how to evaluate the security of any leaking device. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part I*, volume 9056 of *LNCS*, pages 401–429. Springer, Berlin, Heidelberg, April 2015.
- [DvBV23] Jan-Pieter D’Anvers, Michiel van Beirendonck, and Ingrid Verbauwhede. Revisiting Higher-Order Masked Comparison for Lattice-Based Cryptography: Algorithms and Bit-Sliced Implementations. *IEEE Trans. Computers*, 72(2):321–332, 2023.

- [GD23] John Gaspoz and Siemen Dhooghe. Threshold implementations in software: Micro-architectural leakages in algorithms. *IACR TCHES*, 2023(2):155–179, 2023.
- [GHP⁺21] Barbara Gigerl, Vedad Hadzic, Robert Primas, Stefan Mangard, and Roderick Bloem. Coco: Co-design and co-verification of masked software implementations on CPUs. In Michael Bailey and Rachel Greenstadt, editors, *USENIX Security 2021*, pages 1469–1468. USENIX Association, August 2021.
- [GJJR11] Gilbert Goodwill, Benjamin Jun, Josh Jaffe, and Pankaj Rohatgi. A testing methodology for side channel resistance validation. September 2011.
- [GMPP20] Si Gao, Ben Marshall, Dan Page, and Thinh Pham. FENL: an ISE to mitigate analogue micro-architectural leakage. *IACR TCHES*, 2020(2):73–98, 2020.
- [GOP22] Si Gao, Elisabeth Oswald, and Dan Page. Towards micro-architectural leakage simulators: Reverse engineering micro-architectural leakage features is practical. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part III*, volume 13277 of *LNCS*, pages 284–311. Springer, Cham, May / June 2022.
- [GP99] Louis Goubin and Jacques Patarin. DES and differential power analysis (the “duplication” method). In Çetin Kaya Koç and Christof Paar, editors, *CHES’99*, volume 1717 of *LNCS*, pages 158–172. Springer, Berlin, Heidelberg, August 1999.
- [GPM21] Barbara Gigerl, Robert Primas, and Stefan Mangard. Secure and efficient software masking on superscalar pipelined processors. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part II*, volume 13091 of *LNCS*, pages 3–32. Springer, Cham, December 2021.
- [HKŠ] Daniel Heinz, Matthias J. Kannwischer, and Milan Šorf. masked-kyber-m4/mkm4. https://github.com/masked-kyber-m4/mkm4/blob/main/crypto_kem/kyber768/m4/secadd.S. Commit a997a05.
- [HTF09] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer Series in Statistics. Springer, New York, NY, 2 edition, 2009.
- [ISW03] Yuval Ishai, Amit Sahai, and David Wagner. Private circuits: Securing hardware against probing attacks. In Dan Boneh, editor, *CRYPTO 2003*, volume 2729 of *LNCS*, pages 463–481. Springer, Berlin, Heidelberg, August 2003.
- [KDvB⁺22] Suparna Kundu, Jan-Pieter D’Anvers, Michiel van Beirendonck, Angshuman Karmakar, and Ingrid Verbauwhede. Higher-order masked Saber. In Clemente Galdi and Stanislaw Jarecki, editors, *SCN 22*, volume 13409 of *LNCS*, pages 93–116. Springer, Cham, September 2022.
- [KJJ99] Paul C. Kocher, Joshua Jaffe, and Benjamin Jun. Differential power analysis. In Michael J. Wiener, editor, *CRYPTO’99*, volume 1666 of *LNCS*, pages 388–397. Springer, Berlin, Heidelberg, August 1999.
- [KS20] P. Kiaei and P. Schaumont. Domain-oriented masked instruction set architecture for risc-v. In *Proceedings of the Workshop on Secure RISC-V Architecture Design (SECRISCV)*, August 2020.

- [LBS19] Itamar Levi, Davide Bellizia, and François-Xavier Standaert. Reducing a masked implementation’s effective security order with setup manipulations. *IACR TCHES*, 2019(2):293–317, 2019.
- [LKO⁺21] Moritz Lipp, Andreas Kogler, David F. Oswald, Michael Schwarz, Catherine Easdon, Claudio Canella, and Daniel Gruss. PLATYPUS: Software-based power side-channel attacks on x86. In *2021 IEEE Symposium on Security and Privacy*, pages 355–371. IEEE Computer Society Press, May 2021.
- [MM13] Amir Moradi and Oliver Mischke. On the simplicity of converting leakages from multivariate to univariate - (case study of a glitch-resistant masking scheme). In Guido Bertoni and Jean-Sébastien Coron, editors, *CHES 2013*, volume 8086 of *LNCS*, pages 1–20. Springer, Berlin, Heidelberg, August 2013.
- [MPW22] Ben Marshall, Dan Page, and James Webb. MIRACLE: MiCRo-ArChitectural leakage evaluation A study of micro-architectural power leakage across many devices. *IACR TCHES*, 2022(1):175–220, 2022.
- [New18] NewAE Technology Inc. *CW305 Artix FPGA Target Product Datasheet*. NewAE Technology, February 2018. Product datasheet.
- [PV17] Kostas Papagiannopoulos and Nikita Veshchikov. Mind the gap: Towards secure 1st-order masking in software. In Sylvain Guilley, editor, *COSADE 2017*, volume 10348 of *LNCS*, pages 282–297. Springer, Cham, April 2017.
- [SM15] Tobias Schneider and Amir Moradi. Leakage assessment methodology - A clear roadmap for side-channel evaluations. In Tim Güneysu and Helena Handschuh, editors, *CHES 2015*, volume 9293 of *LNCS*, pages 495–513. Springer, Berlin, Heidelberg, September 2015.
- [WPH⁺22] Yingchen Wang, Riccardo Paccagnella, Elizabeth Tang He, Hovav Shacham, Christopher W. Fletcher, and David Kohlbrenner. Hertzbleed: Turning power Side-Channel attacks into remote timing attacks on x86. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 679–697, Boston, MA, August 2022. USENIX Association.
- [ZBPF18] Davide Zoni, Alessandro Barengi, Gerardo Pelosi, and William Fornaciari. A comprehensive side-channel information leakage analysis of an in-order risc cpu microarchitecture. 23(5), August 2018.